

2026년도 전국기능경기대회 채점기준

1. 채점 시 유의사항

직 종 명

클라우드컴퓨팅

※ 다음 사항을 유의하여 채점하십시오.

- 1) AWS 지역은 각 module에서 명시된 Region을 사용합니다.
- 2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다.
- 3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다.
- 4) Shell에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다.
- 5) 문제지와 채점지에 있는 <>는 변수입니다. 해당 부분을 변경해 입력합니다.
- 6) 채점은 문항 순서대로 진행해야 합니다.
- 7) 삭제된 채점자료는 되돌릴 수 없음으로 유의하여 진행하며, 이의신청까지 완료 이후 선수가 생성한 클라우드 리소스를 삭제합니다.
- 8) 부분 점수는 존재하지 않으며, 모든 항목을 정확히 충족해야 점수로 인정됩니다.
- 9) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다.
- 10) (예상 출력)은 바로 이전 (명령어 입력)의 예상 출력을 의미합니다.
- 11) 채점 시에는 별도로 제공한 채점 스크립트들을 실행하여 채점할 수 있습니다.
다만, 선수가 직접 입력을 원할 경우 채점기준표에 명시된 명령어 그대로 입력하여 채점할 수 있습니다.
- 12) 배포된 채점 스크립트 중에서 mark1.sh와 mark3.sh는 EKS를 생성해서 EKS 오브젝트를 채점해야 하므로 Bastion에서 ssh 접속 후 진행합니다.
- 13) 배포된 채점 스크립트 중에서 mark2.sh는 VPC Lattice 채점은 Cloudshell에서 채점이 불가능하므로 Bastion에서 채점을 진행합니다.
- 14) Bastion에 접근 불가시 채점이 불가하므로 반드시 SSH를 통한 접속과 권한이 문제가 없도록 합니다.

2. 채점기준표

1) 주요항목별 배점

직 종 명

클라우드컴퓨팅

과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제2과제	1	EKS Scaling	7.5	○			○	
	2	VPC Lattice	7.5	○			○	
	3	Container Logging	7.5	○			○	
	4	REST API Implement	7.5	○			○	
합 계			30					

2) 채점방법 및 기준

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배점
제2과제	1	EKS Scaling	1	인프라 구성	1.0
			2	EKS 구성	1.0
			3	EKS NodeGroup 구성	1.0
			4	NameSpace 및 deployment 가용성 구성	1.5
			5	KEDA SQS 스케일링 설정	1.5
			6	KEDA 및 Karpenter Scaling 테스트 (수동채점)	1.5
	2	VPC Lattice	1	인프라 구성	1.0
			2	ALB 및 Target Group 구성	1.0
			3	VPC Lattice 구성	1.0
			4	VPC Lattice Default Rule 구성	1.5
			5	VPC Lattice Header Rule 구성	1.5
			6	Application 테스트	1.5
	3	Container logging	1	인프라 구성	1.0
			2	Pod 실행 및 NLB 엔드포인트 구성	1.0
			3	EC2 Docker 컨테이너 실행 테스트	1.0
			4	Fluent Bit 서비스 실행 테스트	1.5
			5	로그 수집 파이프라인 E2E 테스트	1.5
			6	Grafana 대시보드 패널 4종 구성 (수동채점)	1.5
	4	REST API Implement	1	인프라 구성	1.0
			2	API healthcheck 테스트	1.0
			3	API /v1/user 테스트	1.0
			4	Lambda 중복 저장 방지 테스트	1.5
			5	API Key 인증 동작 테스트	1.5
			6	API 요청 차단 확인 테스트	1.5
	합계				

3) 채점 내용

순번	채점 항목	
0	# Bastion 1) 각 모듈에 맞게 구성한 Bastion에 접근합니다. 2) ec2-user로 접근합니다. 3) 아래 파일들을 각 모듈에 맞는 Bastion의 /home/ec2-user/markings/ 디렉터리로 복사합니다. - mark1.sh # EKS Scaling - mark2.sh # VPC Lattice - mark3.sh # Container Logging 4) /home/ec2-user/markings/ 경로에서 스크립트를 실행합니다. 실행 결과를 기반으로 채점을 진행하되 선수가 이익을 제기할 경우 수동으로 채점을 진행할 수 있도록 합니다. 5) 채점을 진행하기 전에 다음 명령어를 수행하여 채점 진행을 위한 사전 작업을 조건에 맞게 진행합니다. (채점 스크립트 진행 시 생략 가능) # CloudShell 1) CloudShell에 접근합니다. 2) cloudshell-user로 접근합니다. 3) 아래 파일들을 CloudShell의 /home/cloudshell-user/markings/ 디렉터리로 복사합니다. - mark4.sh # REST API Implement 4) /home/cloudshell-user/markings/ 경로에서 스크립트를 실행합니다. 실행 결과를 기반으로 채점을 진행하되 선수가 이익을 제기할 경우 수동으로 채점을 진행할 수 있도록 합니다. 5) 채점을 진행하기 전에 다음 명령어를 수행하여 채점 진행을 위한 사전 작업을 조건에 맞게 진행합니다. (채점 스크립트 진행 시 생략 가능)	
	<pre># set default region of aws cli with EKS Scaling aws configure set default.region ap-northeast-2 # set default region of aws cli with VPC Lattice aws configure set default.region ap-southeast-1 # set default region of aws cli with Container logging aws configure set default.region ap-northeast-1 # set default region of aws cli with REST API Implement aws configure set default.region us-east-1</pre>	
1-1	1-1-A	<pre>VPC_ID=\$(aws ec2 describe-vpcs W --region ap-northeast-2 W --filters "Name=tag:Name,Values=wsc-scaling-vpc" W --query "Vpcs[0].VpcId" W --output text) aws ec2 describe-subnets W --region ap-northeast-2 W --filters "Name=vpc-id,Values=\$VPC_ID" W --query "Subnets[*].Tags[?Key=='Name'] [0].Value,AvailabilityZone,CidrBlock" W --output text aws ec2 describe-instances W --region ap-northeast-2 W --filters W "Name=tag:Name,Values=wsc-scaling-bastion" W "Name=instance-state-name,Values=running" W --query "Reservations[*].Instances[*].Tags[?Key=='Name'] [0].Value,InstanceType" W --output text aws sqs list-queues W --query "QueueUrls[?contains(@, 'wsc-scaling-sqs')]" W --output text W awk -F '/' '{print \$NF}'</pre>

순번	채점 항목	
1-1	1-1-A (예상 출력) <u>정확히 일치</u> <u>순서 상관 없음</u>	wsc-scaling-sn-pub-a ap-northeast-2a 10.11.0.0/24 wsc-scaling-sn-pub-c ap-northeast-2c 10.11.1.0/24 wsc-scaling-sn-priv-a ap-northeast-2a 10.11.10.0/24 wsc-scaling-sn-priv-c ap-northeast-2c 10.11.11.0/24 wsc-scaling-bastion t3.medium wsc-scaling-sqs
1-2	1-2-A (명령어 입력)	aws eks describe-cluster W --region ap-northeast-2 W --name wsc-scaling-cluster W --query "cluster.{name:name,version:version}" W --output text
	1-2-A (예상 출력) <u>정확히 일치</u>	wsc-scaling-cluster 1.35
1-3	1-3-A (명령어 입력)	aws eks describe-nodegroup W --region ap-northeast-2 W --cluster-name wsc-scaling-cluster W --nodegroup-name wsc-scaling-node W --query "nodegroup.{nodegroupName:nodegroupName, instanceTypes:join(', ', instanceTypes)}" W --output text aws eks describe-nodegroup W --region ap-northeast-2 W --cluster-name wsc-scaling-cluster W --nodegroup-name wsc-scaling-node W --query "nodegroup.scalingConfig" W --output text tr 'Wt' 'Wn' sort -n aws eks describe-nodegroup W --region ap-northeast-2 W --cluster-name wsc-scaling-cluster W --nodegroup-name wsc-scaling-node W --query "nodegroup.labels.dedicated" W --output text
	1-3-A (예상 출력) <u>정확히 일치</u>	t3.medium wsc-scaling-node 2 2 10 scaling
1-4	1-4-A (명령어 입력)	kubectl get namespace wsc-scaling kubectl get deployment wsc-scaling-deploy -n wsc-scaling kubectl get deployment wsc-scaling-deploy -n wsc-scaling W -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
	1-4-A (예상 출력) <u>빨강색 부분</u> <u>정확히 일치</u>	NAME STATUS AGE wsc-scaling Active 3h42m NAME READY UP-TO-DATE AVAILABLE AGE wsc-scaling-deploy 2/2 2 2 3h41m busybox:latest
1-5	1-5-A (명령어 입력)	kubectl get scaledobject wsc-scaling-scaledobject -n wsc-scaling W -o jsonpath='{.spec.pollingInterval}'; echo kubectl get scaledobject wsc-scaling-scaledobject -n wsc-scaling W -o jsonpath='{.spec.triggers[0].metadata.queueLength}'; echo
	1-5-A (예상 출력) <u>정확히 일치</u>	30 5

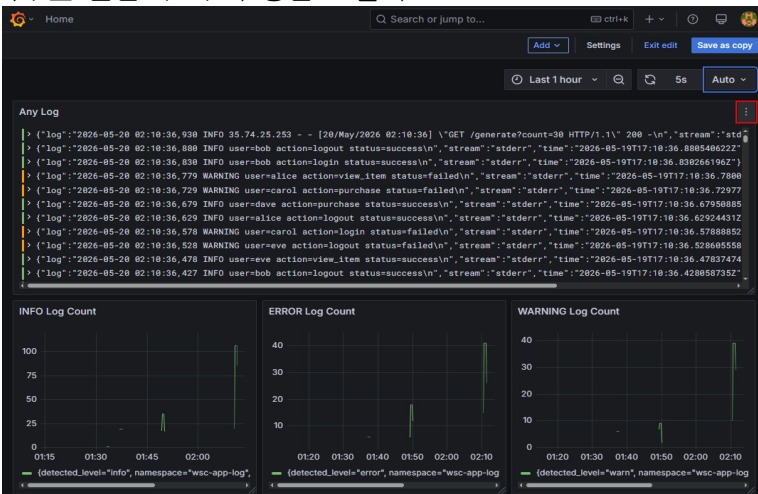
순번	채점 항목	
1-6	1-6-A (명령어 입력)	kubectl get po -n wsc-scaling kubectl get nodes
	1-6-A (예상 출력) 빨간색 부분 정확히 일치	NAME READY STATUS RESTARTS AGE wsc-scaling-deploy-54bbd95c49-d97nd 1/1 Running 0 3m30s wsc-scaling-deploy-54bbd95c49-vkng8 1/1 Running 0 3m30s NAME STATUS ROLES AGE VERSION ip-10-11-10-28.ap-northeast-2.compute.internal Ready <none> 49m v1.35.5-eks-3385e9b ip-10-11-11-218.ap-northeast-2.compute.internal Ready <none> 49m v1.35.5-eks-3385e9b
1-6	1-6-B (명령어 입력) 최대 3분 대기	SQS_URL=\$(aws sqs get-queue-url W --queue-name wsc-scaling-sqs W --query 'QueueUrl' W --output text) for n in {1..100}; do aws sqs send-message W --queue-url "\$SQS_URL" W --message-body "EKS Scaling Test \$n" W > /dev/null echo "EKS Scaling Test: \$n" done sleep 60
1-6	1-6-C (명령어 입력)	kubectl get po -n wsc-scaling kubectl get nodes
	1-6-C (예상 출력) 빨간색 부분 정확히 일치	NAME READY STATUS RESTARTS AGE wsc-scaling-deploy-54bbd95c49-5czt4 1/1 Running 0 12m wsc-scaling-deploy-54bbd95c49-6w6bz 1/1 Running 0 2m38s wsc-scaling-deploy-54bbd95c49-7swlv 1/1 Running 0 2m53s . . . wsc-scaling-deploy-54bbd95c49-znwz5 1/1 Running 0 2m53s (Pod가 19 ~ 20개 있어야 함) Every 2.0s: kubectl get nodes ip-10-11-0-47.ap-northeast-2.compute.internal: Mon May 25 06:48:37 2026 NAME STATUS ROLES AGE VERSION ip-10-11-0-34.ap-northeast-2.compute.internal Ready <none> 7m32s v1.35.5-eks-3385e9b ip-10-11-1-132.ap-northeast-2.compute.internal Ready <none> 8m v1.35.5-eks-3385e9b ip-10-11-10-28.ap-northeast-2.compute.internal Ready <none> 30m v1.35.5-eks-3385e9b ip-10-11-11-218.ap-northeast-2.compute.internal Ready <none> 30m v1.35.5-eks-3385e9b (기존 Node 2개 + 새로 생성된 Node 2개 총 4개가 있어야 함)

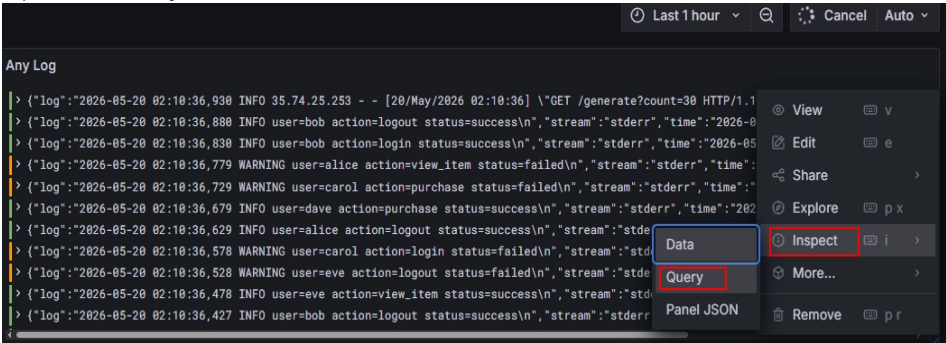
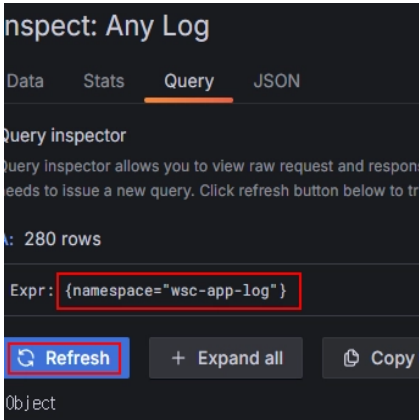
순번	채점 항목			
2-1	2-1-A (명령어 입력)	<pre>aws ec2 describe-vpcs W --filters "Name=tag:Name,Values=wsc-hub-vpc,wsc-spoke-vpc" W --query "Vpcs[*].[Tags[?Key=='Name']][0].Value, CidrBlock]" W --output text aws ec2 describe-subnets W --filters "Name=tag:Name,Values=wsc-hub-sn-pub-a,wsc-hub-sn-pub-c,wsc-spoke-sn-pub-a,wsc-spoke-sn-pub-c,wsc-spoke-sn-priv-a,wsc-spoke-sn-priv-c" W --query "Subnets[*].[Tags[?Key=='Name']][0].Value, CidrBlock]" W --output text aws ec2 describe-instances W --filters "Name=tag:Name,Values=wsc-hub-bastion,wsc-spoke-app-v1,wsc-spoke-app-v2" W --query "Reservations[].Instances[].[Tags[?Key=='Name']][0].Value, InstanceType]" W --output text</pre>		
	2-1-A (예상 출력) 정확히 일치 (순서 상관 없음)	<pre>wsc-hub-vpc 10.0.0.0/16 wsc-spoke-vpc 192.168.0.0/16 wsc-hub-sn-pub-a 10.0.0.0/24 wsc-hub-sn-pub-c 10.0.1.0/24 wsc-spoke-sn-pub-a 192.168.0.0/24 wsc-spoke-sn-pub-c 192.168.1.0/24 wsc-spoke-sn-priv-a 192.168.2.0/24 wsc-spoke-sn-priv-c 192.168.3.0/24 wsc-spoke-app-v1 t3.medium wsc-spoke-app-v2 t3.medium wsc-hub-bastion t3.small</pre>		
2-2	2-2-A (명령어 입력)	<pre>aws elbv2 describe-load-balancers W --names wsc-spoke-app-alb W --query "LoadBalancers[].[LoadBalancerName, Scheme, Type, State.Code]" W --output text aws elbv2 describe-target-groups W --names wsc-spoke-v1-tg wsc-spoke-v2-tg W --query "TargetGroups[].[TargetGroupName, Protocol, Port]" W --output text</pre>		
	2-2-A (예상 출력) 정확히 일치	<pre>wsc-spoke-app-alb internal application active wsc-spoke-v1-tg HTTP 8080 wsc-spoke-v2-tg HTTP 8080</pre>		
2-3	2-3-A (명령어 입력)	<pre>aws vpc-lattice list-service-networks --query "items[?name=='wsc-app-service-network'].name" --output text aws vpc-lattice list-services --query "items[?name=='wsc-app-service'].name" --output text SERVICE_NETWORK_ID=\$(aws vpc-lattice list-service-networks --query "items[?name=='wsc-app-service-network'].id" --output text) aws vpc-lattice list-service-network-vpc-associations --service-network-identifier \$SERVICE_NETWORK_ID --query "items[].vpclId" --output text tr 'Wt' 'Wn' while read vpc_id; do aws ec2 describe-vpcs --vpc-ids \$vpc_id --query "Vpcs[0].Tags[?Key=='Name'].Value" --output text done</pre>		
	2-3-A (예상 출력) 정확히 일치	<pre>wsc-app-service-network wsc-app-service wsc-spoke-vpc wsc-hub-vpc</pre>		

순번	채점 항목	
2-4	2-4-A (명령어 입력)	<pre> SVC_ID=\$(aws vpc-lattice list-services W --query "items[?name=='wsc-app-service'].id" W --output text) LISTENER_ID=\$(aws vpc-lattice list-listeners W --service-identifier "\$SVC_ID" W --query "items[0].id" W --output text) for RULE_ID in \$(aws vpc-lattice list-rules W --service-identifier "\$SVC_ID" W --listener-identifier "\$LISTENER_ID" W --query "items[*].id" W --output text); do RULE=\$(aws vpc-lattice get-rule W --service-identifier "\$SVC_ID" W --listener-identifier "\$LISTENER_ID" W --rule-identifier "\$RULE_ID" W --output json) PRIORITY=\$(echo "\$RULE" jq -r '.priority') if ["\$PRIORITY" != "99999"]; then continue fi HEADER=\$(echo "\$RULE" jq -r '.match.httpMatch.headerMatches[0].match.exact // "default"') TARGETS=\$(echo "\$RULE" W jq -r '.action.forward.targetGroups[] "W(.targetGroupIdentifier):W(.weight)"' W while read -r line; do TG_ID=\$(echo "\$line" cut -d: -f1) WEIGHT=\$(echo "\$line" cut -d: -f2) TG_NAME=\$(aws vpc-lattice get-target-group W --target-group-identifier "\$TG_ID" W --query "name" W --output text) echo -n "\$TG_NAME:\$WEIGHT " done) echo "Priority=\$PRIORITY Header=\$HEADER Targets=\$TARGETS" done </pre>
2-4	2-4-A (예상 출력) 정확히 일치 (순서 상관 없음)	<pre> Priority=99999 Header=default Targets=wsc-spoke-v1-tg:90 wsc-spoke-v2-tg:10 </pre>
2-5	2-5-A (명령어 입력)	<pre> for RULE_ID in \$(aws vpc-lattice list-rules --service-identifier \$SVC_ID --listener-identifier \$LISTENER_ID --query "items[*].id" --output text); do RULE=\$(aws vpc-lattice get-rule --service-identifier \$SVC_ID --listener-identifier \$LISTENER_ID --rule-identifier \$RULE_ID --output json) PRIORITY=\$(echo \$RULE jq -r '.priority') if ["\$PRIORITY" = "99999"]; then continue; fi HEADER=\$(echo \$RULE jq -r '.match.httpMatch.headerMatches[0].match.exact // "default"') TARGETS=\$(echo \$RULE jq -r '.action.forward.targetGroups[] "W(.targetGroupIdentifier):W(.weight)"' while read line; do TG_ID=\$(echo \$line cut -d: -f1) WEIGHT=\$(echo \$line cut -d: -f2) TG_NAME=\$(aws vpc-lattice get-target-group --target-group-identifier \$TG_ID --query "name" --output text) echo -n "\$TG_NAME:\$WEIGHT " done) echo "Priority=\$PRIORITY Header=\$HEADER Targets=\$TARGETS" done </pre>

순번	채점 항목	
2-5	2-5-A (예상 출력) 정확히 일치 (순서 상관 없음)	Priority=20 Header=v2 Targets=wsc-spoke-v2-tg:100 Priority=10 Header=v1 Targets=wsc-spoke-v1-tg:100
2-6	2-6-A (명령어 입력)	<pre>for TG_NAME in wsc-spoke-v1-tg wsc-spoke-v2-tg; do TG_ARN=\$(aws elbv2 describe-target-groups W --names "\$TG_NAME" W --query "TargetGroups[0].TargetGroupArn" W --output text) STATUS=\$(aws elbv2 describe-target-health W --target-group-arn "\$TG_ARN" W --query "TargetHealthDescriptions[0].TargetHealth.State" W --output text) echo "\$TG_NAME \$STATUS" done SVC_ID=\$(aws vpc-lattice list-services W --query "items[?name=='wsc-app-service'].id" W --output text) LATTICE_DNS=\$(aws vpc-lattice get-service W --service-identifier "\$SVC_ID" W --query "dnsEntry.domainName" W --output text) curl -s -H "version: v1" http://\$LATTICE_DNS/version curl -s -H "version: v2" http://\$LATTICE_DNS/version</pre>
	2-6-A (예상 출력) 정확히 일치	<pre>wsc-spoke-v1-tg healthy wsc-spoke-v2-tg healthy {"version":"v1"} {"version":"v2"}</pre>
3-1	3-1-A (명령어 입력)	<pre>aws ec2 describe-vpcs W --region ap-northeast-1 W --filters "Name=tag:Name,Values=wsc-log-vpc" W --query "Vpcs[0].CidrBlock" --output text aws ec2 describe-subnets W --region ap-northeast-1 W --filters "Name=tag:Name,Values=wsc-logging-sn-pub-a,wsc-logging-sn-pub-c,wsc-logging-sn-priv-a,wsc-logging-sn-priv-c" W --query "Subnets[*].[Tags[?Key=='Name'].Value [0],CidrBlock]" W --output text aws eks describe-cluster W --region ap-northeast-1 W --name wsc-logging-cluster W --query "cluster.[name,version]" W --output text aws eks describe-nodegroup W --region ap-northeast-1 W --cluster-name wsc-logging-cluster W --nodegroup-name wsc-logging-ng W --query "nodegroup.[nodegroupName,scalingConfig]" W --output json</pre>

순번	채점 항목	
3-1	3-1-A (예상 출력) <u>빨강색 부분</u> <u>정확히 일치</u> (순서 상관 없음)	<pre> 10.3.0.0/16 wsc-logging-sn-pub-a 10.3.0.0/24 wsc-logging-sn-priv-a 10.3.2.0/24 wsc-logging-sn-pub-c 10.3.1.0/24 wsc-logging-sn-priv-c 10.3.3.0/24 wsc-logging-cluster 1.35 ["wsc-logging-ng", { "minSize": 2, "maxSize": 4, "desiredSize": 2 }]</pre>
3-2	3-2-A (명령어 입력)	<pre> kubectl get pods -n wsc-logging -l app.kubernetes.io/name=loki,app.kubernetes.io/component=single-binary awk '{ print \$3 }' kubectl get svc -n wsc-logging -l app.kubernetes.io/name=loki grep LoadBalancer awk '{ print \$4 }' kubectl get pods -n wsc-logging -l app.kubernetes.io/name=grafana awk '{ print \$3 }' kubectl get svc -n wsc-logging -l app.kubernetes.io/name=grafana grep LoadBalancer awk '{ print \$4 }'</pre>
	3-2-A (예상 출력) <u>빨강색 부분</u> <u>정확히 일치</u>	<pre> STATUS Running a11c8436181b54e69aa5f34ec1e16b62-733c9ef8646d3165.elb.ap-northeast-1.amazonaws.com STATUS Running a2c92ca29c5894aacb5662f1e6e3710f-92fe7d5e9f9ba881.elb.ap-northeast-1.amazonaws.com</pre>
3-3	3-3-A (명령어 입력)	<pre> EC2_IP=\$(aws ec2 describe-instances W --region ap-northeast-1 W --filters "Name=tag:Name,Values=wsc-logging-app-bastion" W "Name=instance-state-name,Values=running" W --query "Reservations[0].Instances[0].PublicIpAddress" --output text) curl -s "http://\$EC2_IP:5000/health"</pre>
	3-3-A (예상 출력) <u>정확히 일치</u>	<pre> {"status":"ok"}</pre>
3-4	3-4-A (명령어 입력)	<pre> INSTANCE_ID=\$(aws ec2 describe-instances W --region ap-northeast-1 W --filters "Name=tag:Name,Values=wsc-logging-app-bastion" W --query "Reservations[0].Instances[0].InstanceId" --output text) CMD_ID=\$(aws ssm send-command W --region ap-northeast-1 W --instance-ids \$INSTANCE_ID W --document-name "AWS-RunShellScript" W --parameters '{"commands":["systemctl is-active fluent-bit"]}' W --query "Command.CommandId" --output text) sleep 5 aws ssm get-command-invocation W --region ap-northeast-1 W --command-id \$CMD_ID --instance-id \$INSTANCE_ID W --query "StandardOutputContent" --output text</pre>

순번	채점 항목	
3-4	3-4-A (예상 출력) <u>정확히 일치</u>	active
3-5	3-5-A (명령어 입력) 10초 대기후 3-5-B 명령어 입력	EC2_IP=\$(aws ec2 describe-instances W --region ap-northeast-1 W --filters "Name=tag:Name,Values=wsc-logging-app-bastion" W "Name=instance-state-name,Values=running" W --query "Reservations[0].Instances[0].PublicIpAddress" --output text) sleep 10 curl -s "http://\$EC2_IP:5000/generate?count=30" > /dev/null 2>&1
	3-5-B (명령어 입력)	LOKI_LB=\$(kubectl get svc -n wsc-logging -l app.kubernetes.io/name=loki grep LoadBalancer awk '{ print \$4 }') curl -sG "http://\$LOKI_LB:3100/loki/api/v1/query_range" W --data-urlencode 'query={namespace="wsc-app-log"}' W --data-urlencode 'limit=10' W python3 -c "import sys,json; d=json.load(sys.stdin); print(len(d.get('data',{}).get('result',[])), 'streams found')"
	3-5-B (예상 출력) <u>정확히 일치</u>	3 streams found <- 1개 이상 해야 함
3-6	3-6-A (명령어 입력) 비번호를 입력하시오	GRAFANA_LB=\$(kubectl get svc -n wsc-logging -l app.kubernetes.io/name=grafana grep LoadBalancer awk '{ print \$4 }') read -s -p "비번호를 입력하시오: " NM curl -s -u wsc2026-admin-\$NM:admin\$NM! W "http://\$GRAFANA_LB/api/search?type=dash-db" W python3 -c "import sys,json; print(len(json.load(sys.stdin)), 'dashboards')"
	3-6-A (예상 출력) <u>정확히 일치</u>	1 dashboards <- 1개 이상 해야함
	3-6-B 브라우저 접속	kubectl get svc -n wsc-logging -l app.kubernetes.io/name=grafana grep LoadBalancer awk '{ print \$4 }'에 출력된 값을 복사합니다. Grafana(http://<위에서 출력된 DNS 값>)으로 접속합니다. 아이디는 wsc2026-admin-{비번호}를 입력하며, 비밀번호는 admin{비번호}!를 사용합니다. 왼쪽 Dashboards 탭을 눌러 WSC2026 Container Logs에 접근합니다. 아래 4종 패널이 모두 구성이 되었는지 확인합니다. 각 패널 마우스 올린 후 우측 상단 : 클릭 

순번	채점 항목	
	3-6-B query 테스트	Inspect > Query 클릭 
3-6	3-6-B query 테스트	Log Query 확인  총 4개의 Query 확인 {namespace="wsc2026-app-log"} 로그 스트림 출력 확인 count_over_time({namespace="wsc-app-log"} = "INFO" [1m]) 시계열 그래프 출력 확인 count_over_time({namespace="wsc-app-log"} = "ERROR" [1m]) 시계열 그래프 출력 확인 count_over_time({namespace="wsc-app-log"} = "WARNING" [1m]) 시계열 그래프 출력 확인
	3-6-C (명령어 입력) 최대 15초 대기	<pre>EC2_IP=\$(aws ec2 describe-instances --region ap-northeast-1 --filters "Name=tag:Name,Values=wsc-log-app-bastion" "Name=instance-state-name,Values=running" --query "Reservations[0].Instances[0].PublicIpAddress" --output text) curl -s "http://\$EC2_IP:5000/" curl -s "http://\$EC2_IP:5000/health" curl -s "http://\$EC2_IP:5000/generate?count=30" jq grep generated</pre>
	3-6-C (예상 출력) 정확히 일치	<pre>{"service": "m3-log-generator", "status": "healthy"} {"status": "ok"} "generated": 30,</pre>

순번	채점 항목	
4-1	4-1-A	<pre>aws apigateway get-rest-apis W --query "items[?name=='wsc-rest-api'].name" W --output text aws apigateway get-stages W --rest-api-id \$(aws apigateway get-rest-apis W --query "items[?name=='wsc-rest-api'].id" W --output text) W --query "item[?stageName=='prod'].stageName" W --output text aws lambda get-function-configuration W --function-name wsc-rest-function W --query '{LambdaName:FunctionName}' W --output text aws lambda get-function-configuration W --function-name wsc-rest-function W --query 'Runtime' W --output text aws dynamodb list-tables W --query "TableNames[?@=='wsc-rest-table']" W --output text</pre>
	4-1-A (예상 출력) <u>정확히 일치</u>	<pre>wsc-rest-api prod wsc-rest-function python3.14 wsc-rest-table</pre>
4-2	4-2-A (명령어 입력)	<pre>API_URL=\$(aws apigateway get-rest-apis --region us-east-1 --query 'items[?name=='wsc-rest-api'].id' --output text) API_KEY=\$(aws apigateway get-api-keys --region us-east-1 --include-values --query 'items[?name=='wsc-rest-api-key'].value' --output text)</pre> <pre>curl https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/healthcheck</pre>
	4-2-A (예상 출력) <u>정확히 일치</u>	<pre>{"status":"ok"}</pre>
4-3	4-3-A (명령어 입력)	<pre>API_KEY=\$(aws apigateway get-api-keys W --region us-east-1 W --include-values W --query "items[?name=='wsc-rest-api-key'].value" W --output text) aws dynamodb scan --table-name wsc-rest-table --projection-expression "#n" --expression-attribute-names '{"#n":"name"}' jq -c '.Items[]' while read item; do aws dynamodb delete-item --table-name wsc-rest-table --key "\$item" > /dev/null done curl -X POST https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/user W -H "x-api-key: \$API_KEY" W -H "Content-Type: application/json" W -d '{"name": "kim", "age": 19, "country": "korea"}' curl "https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/user?name=k im&age=19" W -H "x-api-key: \$API_KEY"</pre>

순번	채점 항목	
4-3	4-3-A (예상 출력) <u>정확히 일치</u>	<pre>{"message": "User created successfully"} {"name": "kim", "country": "korea", "age": 19}</pre>
4-4	4-4-A (명령어 입력)	<pre>curl -X POST https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/user W -H "x-api-key: \$API_KEY" W -H "Content-Type: application/json" W -d '{"name": "kim", "age": 19, "country": "korea"}' curl "https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/user?name=nobody&age=19" W -H "x-api-key: \$API_KEY"</pre>
	4-4-A (예상 출력) <u>정확히 일치</u>	<pre>{"message": "User already exists"} {"message": "User not found"}</pre>
4-5	4-5-A (명령어 입력)	<pre>curl -X POST https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/user W -H "Content-Type: application/json" W -d '{"name": "kim", "age": 19, "country": "korea"}'</pre>
	4-5-A (예상 출력) <u>정확히 일치</u>	<pre>{"message": "Forbidden"}</pre>
4-6	4-6-A <u>명령어 입력</u>	<pre>curl "https://\$API_URL.execute-api.us-east-1.amazonaws.com/prod/v1/user?name=nobody" W -H "x-api-key: \$API_KEY"</pre>
	4-6-A (예상 출력) <u>정확히 일치</u>	<pre>{"message": "Missing required request parameters: [age]"}</pre>